

Instituto Superior Tecnológico Sudamericano

MANUAL DE PROGRAMADOR

SINKA - Guía Técnica de Desarrollo

Autor:

David Calleh

Director:

Ing. Cristhian Javier Villamarin Gaona

Loja - Ecuador

2025

Índice

- 1 Stack Tecnológico
 - 1.1 Frontend
 - 1.2 Backend
 - 1.3 Base de Datos
 - 1.4 Tiempo Real
- 2 Arquitectura
 - 2.1 Visión General
 - 2.2 Componentes
 - 2.3 Flujos de Datos
- 3 Instalación
 - 3.1 Requisitos Previos
 - 3.2 Setup del Backend
 - 3.3 Setup del Frontend
 - 3.4 Variables de Entorno
- 4 Backend - FastAPI
 - 4.1 Estructura de Carpetas
 - 4.2 Modelos de Datos
 - 4.3 Endpoints REST
 - 4.4 WebSocket
 - 4.5 Servicios
- 5 Frontend - Next.js
 - 5.1 Estructura de Proyecto
 - 5.2 Páginas y Rutas
 - 5.3 Componentes
 - 5.4 Hooks Personalizados
 - 5.5 Gestión de Estado
- 6 Base de Datos
 - 6.1 Esquema
 - 6.2 Migraciones
 - 6.3 Queries Comunes
- 7 Autenticación
 - 7.1 JWT
 - 7.2 Refresh Tokens
 - 7.3 Seguridad
- 8 Despliegue
 - 8.1 Frontend en Vercel
 - 8.2 Backend en Railway
 - 8.3 CI/CD
- 9 Extensión y Customización
- 10 Troubleshooting

1. Stack Tecnológico

1.1 Frontend

Framework: Next.js 14 con App Router para renderizado servidor/cliente hybrid.

Lenguaje: TypeScript para type safety en JavaScript.

Estilos: Tailwind CSS para utilidades CSS prefabricadas.

Animaciones: Framer Motion para transiciones fluidas.

Estado: Zustand para gestión de estado global minimalista.

Iconos: Lucide React para SVG iconografía vectorial.

1.2 Backend

Framework: FastAPI (Python) para API REST rápida con validación automática.

ORM: SQLAlchemy para mapeo objeto-relacional de datos.

Migraciones: Alembic para versionado de esquema de BD.

Validación: Pydantic para esquemas de datos tipados.

Async: Soporta operaciones asincrónicas nativas.

1.3 Base de Datos

Sistema: PostgreSQL relacional con soporte para JSON, arrays, y tipos custom.

Conexiones: Connection pooling para eficiencia.

Backups: Respallos automáticos cada 24 horas.

1.4 Tiempo Real

WebSocket: Comunicación bidireccional para eventos en vivo.

WebRTC: Simple-peer para P2P video/audio entre navegadores.

Signaling: Relay de señales WebRTC a través del servidor.

2. Arquitectura

2.1 Visión General

SINKA sigue una arquitectura cliente-servidor con componentes de tiempo real: **Cliente (Frontend)**: Next.js renderiza UI reactiva. Comunica con backend vía REST API para datos persistentes y WebSocket para eventos en vivo. **Servidor (Backend)**: FastAPI maneja autenticación, persistencia de datos, lógica de negocio, y relay de mensajes WebSocket. **Datos**: PostgreSQL almacena usuarios, sesiones, estadísticas, configuración.

2.2 Componentes

Auth: JWT tokens con refresh automático. **Matching**: Algoritmo de emparejamiento por afinidad. **Sessions**: Orquestación de sesiones de concentración. **Gamification**: Cálculo de XP, monedas, rachas. **WebRTC**: Signaling y relay de pares.

2.3 Flujos de Datos

1. Usuario login → JWT token → Frontend almacena en cookie. 2. Usuario busca sesión → API POST /sessions → Matching engine busca pareja. 3. Pareja encontrada → WebSocket notifica a ambos → Inicio de sesión. 4. Durante sesión → WebRTC establece P2P audio/video → WebSocket transporta señales. 5. Fin de sesión → API POST /sessions/{id}/complete → Cálculo de recompensas → BD actualiza stats.

3. Instalación

3.1 Requisitos Previos

Node.js: v18 o superior **Python:** v3.10 o superior **PostgreSQL:** v12 o superior **Git:** para control de versiones **npm/pip:** gestores de paquetes **Vercel CLI:** para despliegue frontend (opcional)

3.2 Setup del Backend

```
git clone cd backend python -m venv venv source venv/bin/activate (o: .\venv\Scripts\activate en Windows) pip install -r requirements.txt alembic upgrade head uvicorn main:app --reload
```

3.3 Setup del Frontend

```
cd ../frontend npm install npm run dev
```

3.4 Variables de Entorno

Backend (.env): DATABASE_URL=postgresql://... SECRET_KEY= JWT_SECRET= FRONTEND_URL=http://localhost:3000 **Frontend (.env.local):** NEXT_PUBLIC_API_URL=http://localhost:8000

4. Backend - FastAPI

4.1 Estructura de Carpetas

/backend ■■■■ /core - Configuración central ■ ■■■■ config.py ■ ■■■■ security.py ■ ■■■■ ws_auth.py
■■■■ /models - SQLAlchemy ORM ■ ■■■■ user.py ■ ■■■■ session.py ■ ■■■■ ... ■■■■ /schemas -
Pydantic DTOs ■■■■ /services - Lógica de negocio ■■■■ /routers - Endpoints API ■■■■ /database -
Conexión BD ■■■■ main.py - Punto de entrada ■■■■ requirements.txt

4.2 Modelos de Datos

User: id, email, password_hash, username, level, xp, coins, avatar_symbol, areas, created_at
Session: id, initiator_id, partner_id, area, start_time, end_time, duration, xp_earned **UserStats:**
user_id, sessions_completed, streak, max_streak, total_xp **ShopItem:** id, name, type, price,
description **UserInventory:** user_id, item_id, acquired_at

4.3 Endpoints REST

Auth: POST /auth/register POST /auth/login POST /auth/refresh **Users:** GET /users/{id} PUT
/users/{id} GET /users/me **Sessions:** POST /sessions GET /sessions/{id} POST
/sessions/{id}/complete GET /sessions (historial) **Shop:** GET /shop/items POST /shop/buy GET
/shop/inventory

4.4 WebSocket

Endpoint: /ws/session/{session_id} Autenticación: JWT en query param Mensajes: JSON con type,
data Eventos: session.start, session.message, session.voice_signal, session.end

4.5 Servicios

MatchmakingService: Encuentra pareja ideal. **SessionService:** Orquesta sesiones.
GamificationService: Calcula XP, monedas, rachas. **GardenService:** Evoluciona jardín visual.
ShopService: Gestiona tienda.

5. Frontend - Next.js

5.1 Estructura de Proyecto

/frontend ■■■■ /app - App Router (páginas) ■ ■■■■ (auth) ■ ■ ■■■■ login ■ ■ ■■■■ register ■ ■ ■■■■
onboarding ■ ■■■■ dashboard ■ ■■■■ session/[id] ■ ■■■■ leaderboard ■ ■■■■ layout.tsx ■■■■
/components - Componentes reutilizables ■■■■ /hooks - Custom React hooks ■■■■ /store - Zustand
state ■■■■ /styles - Tailwind globals ■■■■ package.json

5.2 Páginas y Rutas

/ → Landing (si no autenticado) o /dashboard (si autenticado) /login → Autenticación /register →
Registro /onboarding → Setup de perfil /dashboard → Panel principal /session/[id] → Sesión activa
/leaderboard → Rankings globales /profile → Mi perfil /shop → Tienda

5.3 Componentes

Navbar: Navegación y usuario menu **Dashboard:** Panel principal **SessionCard:** Preview de sesión
SessionInterface: Pantalla de sesión **ChatBox:** Mensajes en vivo **TimerWidget:** Contador Pomodoro
VideoBox: Render de video WebRTC

5.4 Hooks Personalizados

useAuth() - Autenticación y JWT useSession() - Estado de sesión actual useWebSocket() - Conexión
WS useWebRTC() - Manejo de pares P2P useLocalStorage() - Persistencia cliente useUser() - Datos
del usuario actual

5.5 Gestión de Estado

Zustand store estructura: authStore - JWT, user data, login/logout sessionStore - current session,
partner data, messages uiStore - tema, idioma, notificaciones Actions: login(), logout(),
createSession(), sendMessage(), updateXP()

6. Base de Datos

6.1 Esquema

13 tablas principales: users, user_stats, sessions, matches, shop_items, user_inventory, friends, groups, group_members, gamification_events, audit_log, refresh_tokens, notifications. Relaciones: users 1:N sessions, users 1:1 user_stats, sessions M:M shop_items (inventory).

6.2 Migraciones

Ubicación: /backend/alembic/versions/ Nomenclatura: 0001_initial.py, 0002_add_shops.py, etc. Para crear: alembic revision --autogenerate -m 'descripcion' Para aplicar: alembic upgrade head Para revertir: alembic downgrade -1

6.3 Queries Comunes

Obtener usuario: db.query(User).filter(User.id==id).first() Crear sesión: db.add(Session(...)); db.commit() Actualizar XP: user.xp += amount; db.commit() Buscar pareja: filter por area, level, disponibilidad Calcular racha: COUNT sesiones consecutivas by fecha

7. Autenticación

7.1 JWT

Access Token: Válido 15 minutos, contiene user_id y scopes. Refresh Token: Válido 30 días, usado para obtener nuevo access token. Almacenamiento frontend: Access en memory, Refresh en httpOnly cookie (seguro).

7.2 Refresh Tokens

Endpoint: POST /auth/refresh Envía refresh token en body Devuelve nuevo access token Si refresh expiró: requiere login nuevamente

7.3 Seguridad

Contraseñas: Hash bcrypt con salt. Tránsito: HTTPS/TLS obligatorio. CSRF: Tokens CSRF en formularios. Rate limiting: 100 requests/min por IP en /auth. Validación entrada: Pydantic valida todos los DTOs.

8. Despliegue

8.1 Frontend en Vercel

1. Push a GitHub 2. Conecta repo en Vercel (settings → GitHub) 3. Configura variable NEXT_PUBLIC_API_URL 4. Vercel detecta /frontend y configura build automático 5. Deploy automático en cada push a main

8.2 Backend en Railway

1. Crear proyecto en Railway 2. Conectar GitHub repo 3. Configurar build command: `pip install -r requirements.txt` 4. Configurar start command: `uvicorn main:app --host 0.0.0.0 --port $PORT` 5. Agregar variables de entorno en Railway dashboard 6. Deploy automático en cada push

8.3 CI/CD

GitHub Actions corre tests en cada push. Linter y type checker validan código. Tests unitarios verifican servicios críticos. Solo deploy si tests pasan. Rollback automático si deploy falla.

9. Extensión y Customización

Para agregar funcionalidad nueva: 1. Backend: Crea modelo en /models, schema en /schemas, servicio en /services, endpoints en /routers 2. BD: Crea migración Alembic 3. Frontend: Crea componente en /components, página en /app, hook si necesita estado 4. Estado: Actualiza Zustand store si necesita estado global 5. Tests: Crea test unitario para lógica crítica 6. Despliegue: Git push dispara CI/CD automático

10. Troubleshooting

CORS error en frontend: Verifica FRONTEND_URL en backend .env **WebSocket no conecta:** Revisa autenticación JWT en ws_auth.py **Sesión se desconecta:** Aumenta timeout en config.py **BD conexión falla:** Verifica DATABASE_URL y firewall **Build falla:** Revisa logs de Vercel/Railway en dashboard